

Tutorato di Algoritmi e Laboratorio 02

Ripassiamo un po'... Cos'è...

1) Un albero binario?

- . albero vuoto
- { . radice
- . nodo con al più due figli (alberi binari)

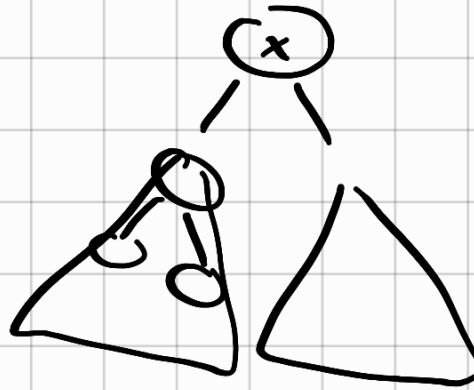
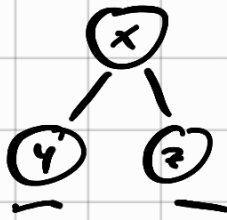
2) Un albero binario completo?

Tutte le foglie hanno la stessa profondità:

Profondità : $h(\text{root}) = 0$
 $h(\text{node}) = h(\text{parent}(\text{node})) + 1$

3) Un albero binario di ricerca?

4) un heap binario?



Domanda: un heap è una struttura ricorsiva?

Come sfruttare questa proprietà?

Domanda ... come possono essere violate le
proprietà dell' heap?

Ecco come ragioneremo:

Operazione: NO?E?

Hp: CHE CONDIZIONI VALGONO AL NOIENTO DELLA
APPLICAZIONE DELL'OPERAZIONE?

Idea: [...]

Pseudocodice: [...]

Complessità: [...]

Operazione: Inserimento di n in H

H_p : H è un heap valido



Incontrano un problema

$$\cdot x < y$$

Non incontrano un problema

$$\cdot x > y$$

$\therefore x$ non ha padre

INSERT-KEY ($H, m, x, \cdot$)

$K[x]$ chiave di x

$m \leftarrow m+1$

$H[m] \leftarrow x$

DOWN (H, m)

DOWN ($H, m, \cdot$)

IF ($m == 1$) RETURN

$p \leftarrow \text{PARENT}(m)$

IF ($H[m] > H[p]$)

SWAP (p, m)

DOWN ($H, p, \cdot$)

ITERATIVE_INSERT ($H, m, x, \cdot$)

WHILE ($m \neq 1 \wedge H[m] > H(\text{PARENT}(m))$)

SWAP ($\text{parent}(m), m$)

$m \leftarrow \text{parent}(m)$

$\begin{cases} m \leftarrow m+1 \\ H[m] \leftarrow x \end{cases}$

Operazione : Decrease-Key

Hp:

DECREASE ($i, x, H, \cdot$)

$H[i] \leftarrow x$

DOWN ($i, H, \cdot$)

Operazione: **Extract-min**

H_p : " "

Pseudo:

EXTRACT min ($H, n, \cdot$)

$R \leftarrow H[1]$

IF ($n \neq 1$)

$H[1] \leftarrow H[n]$

$n \leftarrow n - 1$

IF ($n \neq 0$)

HEAPIFY ($H, n, 1$)

RETURN R

HEAPIFY (H, n, i)

$min \leftarrow i$

IF ($\text{left}(i) \leq n \wedge H(\text{left}(i)) < H(min)$)

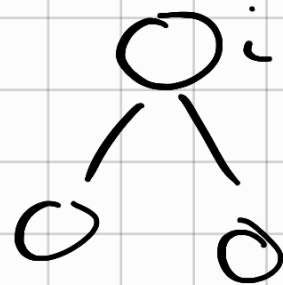
$min \leftarrow \text{left}(i)$

IF (analogo con right)

IF ($i \neq min$)

SWAP (i, min)

HEAPIFY (min)



Operazione: Delete

H_p:

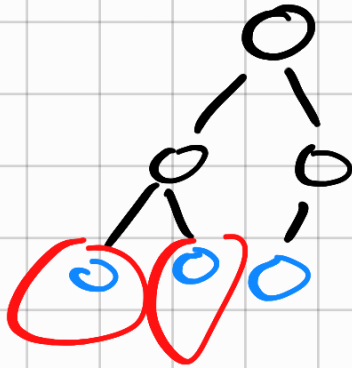
DELETE (H, m, i, .)

DECREASE (H, m, i, $-\infty$, .)

EXTRACT MIN (. . .)

Operations: Naive Build-Heap

Hp:



BUILD-HEAP ($H, n, \cdot$)

FOR $i \leftarrow \lfloor \frac{n}{2} \rfloor$ DOWN TO 1

HEAPIFY ($H, n, i, \cdot$)

Operazione : Build-Heap

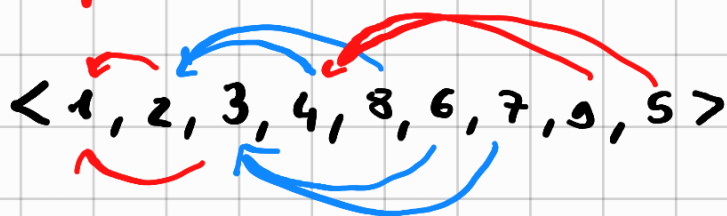
Hp:

Esercizio: Illustrare

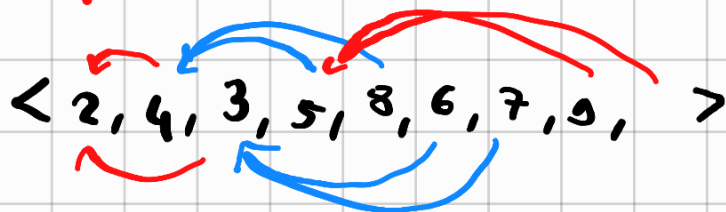
$v: \langle 9, 4, 7, 1, 8, 6, 3, 2, 5 \rangle$

operazioni: build-min-heap
extract-min
extract-min
extract-min
insert(2)
insert(3)
insert(1)
extract-min
extract-min
extract-min

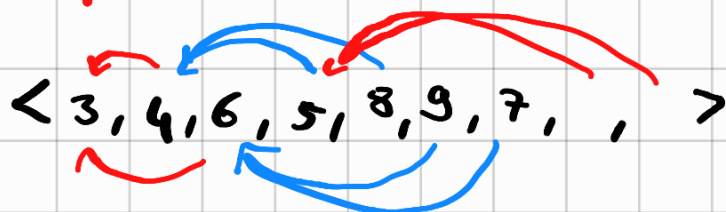
Operazione 1



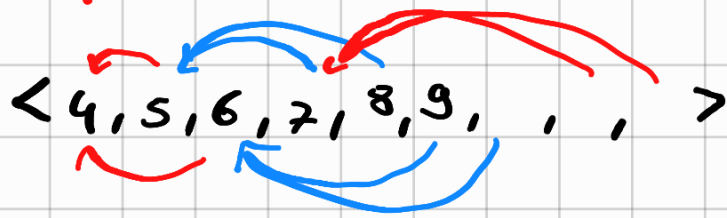
Operazione 2



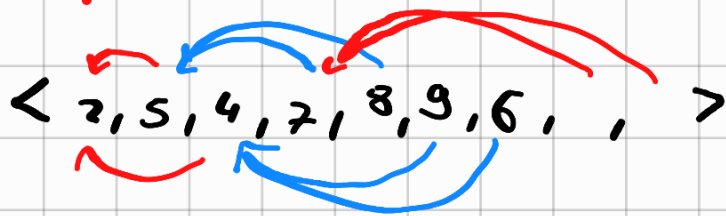
Operazione 3



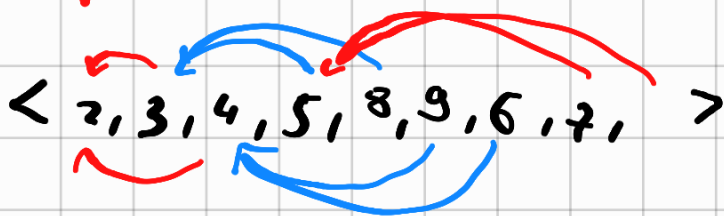
Operazione 4



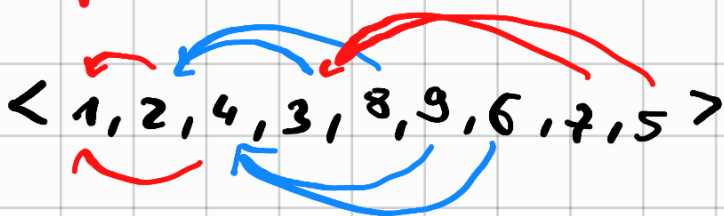
Operazione 5



Operazione 6



Operazione 7



Troppo Bravi 🏆 🏆 🏆

Adesso Spiegate mi Heapsort!

Convincetemi che funziona!